

操作系统综合专题：内核状态机与跨子系统推演

用对象、关系、队列、事件与约束推演跨子系统过程

408 操作系统综合手册

核心模型：状态、转换与评价

进程、虚拟内存、并发、I/O 和文件系统专题分别定义局部机制；这里仅追踪多个机制参与同一过程时的状态变化。

分析框架：

State：系统现在有什么

Transition：系统怎样变化

Evaluation：哪些变化合法、哪些更好

其中：

$S = (O, R, Q)$

分别表示对象 Objects、关系 Relations、队列 Queues；状态变化由事件 Event、机制 Mechanism、策略 Policy 共同决定；所有变化必须满足不变量 Invariants，并承担一定成本 Cost。

概念归属

元概念	唯一归属	不再重复出现为什么成立
Naming / Indirection	State 中的 Relations	它描述“谁通过什么名字/映射指向谁”，不是独立运行规律
Protection / Isolation	Evaluation / Invariants	它回答“哪些状态/操作不允许出现”，不是第二套状态结构
Mechanism / Policy	Transition	机制定义可发生的转换，策略只在合法转换中选择
Scarcity / Arbitration	Queue + Policy	稀缺表现为请求进入队列；仲裁就是策略选谁获得资源
Consistency / Recovery	Invariant + Repair	一致性是不变量；恢复是故障后重新建立不变量的转换
Fast / 慢路径	Cost Optimization	它只回答“怎样把常见情况做便宜”，不再作为运行定律
Event-driven Model	Transition 的驱动力	Event 是状态变化的触发输入，本身不再另设一层
Lifetime / Ownership	O/R 演化 + 生命周期约束	对象何时销毁取决于引用关系与生命周期不变量
Caching / Laziness	Cost Optimization	是利用局部性和推迟工作来降低成本的技术

目录

第 0 章：删掉章节名以后，操作系统还剩下什么

操作系统不是五套机制的拼盘

如果把“进程管理、内存管理、并发、I/O、文件系统”这些教材章节名全部删掉，内核仍然在持续做同一种工作：

1. 维护一组对象以及对象之间的关系；
2. 接收来自程序、硬件、时间和内部子系统的事件；
3. 依据硬件机制和内核机制改变系统状态；
4. 当存在多个合法选择时，根据策略决定下一步；
5. 保证新的状态仍满足安全、隔离、进展、一致性与生命周期要求；
6. 在正确性的约束下尽量降低延迟、空间、I/O、竞争和恢复成本。

因此本册的总模型不是一句口号，而是一套可直接用于推题的形式化描述。

统一状态转移模型

设内核当前状态为

$$S_t = (O_t, R_t, Q_t),$$

其中：

- O_t ：当前存在的内核对象及其内部状态；
- R_t ：对象之间的引用、映射、所有权与命名关系；
- Q_t ：由于资源稀缺、等待条件或异步请求形成的队列。

某个事件 E_t 到来以后，机制 M 给出此时**允许发生**的动作集合：

$$\mathcal{A}_t = M(S_t, E_t).$$

如果存在多个合法动作，策略 π 选择其中一个：

$$a_t = \pi(S_t, E_t, \mathcal{A}_t).$$

随后状态转换：

$$S_{t+1} = T(S_t, E_t, a_t).$$

但不是任何 S_{t+1} 都允许存在，必须满足：

$$I(S_{t+1}) = \text{true},$$

并在满足正确性的前提下优化成本：

$$\min C(S_t, E_t, a_t).$$

一句话总模型

OS = State + Event + Allowed Transition + Policy + Invariant + Cost

进程、页表、锁、fd、inode、设备都只是 O 中不同类型的对象；地址映射、fd 引用、路径解析都只是 R 中不同类型的关系；run queue、wait queue、I/O queue 都只是 Q 的不同实例。

三维模型为什么不会重复

维度	唯一问题	典型关键词
State	现在有什么？彼此怎么连接？谁在等？	object, reference, mapping, queue, owner, state
Transition	什么事件来了？允许怎么改？实际选哪种改法？	event, mechanism, policy, wakeup, fault, interrupt
Evaluation	哪些状态绝不能出现？怎样更快/更省？	protection, invariant, liveness, consistency, cache, 快路径

这三维不存在“同一概念换名字再讲一遍”的必要：**保护**永远是合法性约束；**仲裁**永远是有队列时的策略选择；**间接层**永远是对象关系的一种表示。

Part I: State——内核现在到底保存了什么

第 1 章：Objects——内核首先是对象管理器

对象的统一定义

一个内核对象不是“一个结构体名字”，而是内核需要长期追踪的逻辑实体。任何对象都可以先用三元组理解：

$$Object = (Identity, State, Lifetime).$$

对象	Identity	State	Lifetime 关键事件
Task/Thread	PID/TID/内核引用	Running / Ready / Blocked；寄存器上下文	create / exit / wait / reap
Address Space	mm/VMA 集合	映射、权限、驻留状态	fork / exec / unmap / exit
Physical Page	PFN/页对象	clean / dirty / referenced / pinned	alloc / reclaim / free
Open File	内核 file object	offset、flags、引用计数	open / dup / close
Inode	文件系统对象身份	size、mode、mapping、links	create / link / unlink / evict
Lock/Sync Object	锁对象地址/句柄	owner、waiters、计数	init / acquire / release / destroy
Device/Request	device/request identity	pending / in-flight / done / error	submit / complete / retry

跨专题第一问

不要先问“这是哪一章的题”。先列出**本题真正存在的对象**。一旦对象列错，后面的状态、引用、等待和生命周期一定会错。

对象状态与对象身份必须分开

一个对象可以改变状态，却仍然是同一个对象：

$$Task : Running \longrightarrow Blocked \longrightarrow Ready.$$

一个名字也可以改变，却不改变对象：

$$\text{rename}(\text{pathA}, \text{pathB})$$

改变命名关系，但 inode 对象可以保持不变。

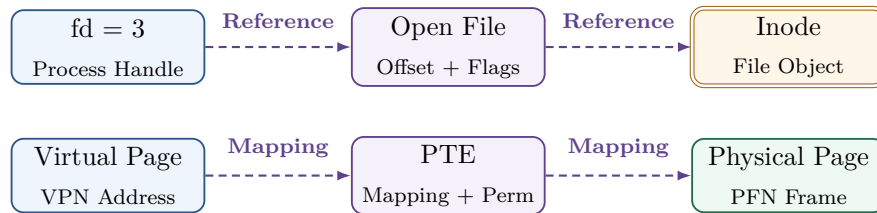
这就是为什么本册把 **Name** 从 Object 中剥离：名字是引用入口，不是对象本身。

第 2 章：Relations——Naming、Reference、Mapping 都只是“边”

统一关系图

对象之间最重要的不是“层数”，而是**边的语义**。典型关系如下：

$$Source\ Object/Name \xrightarrow{relation} Target\ Object.$$



Naming & Indirection 的唯一位置

Naming & Indirection 在本册中不再作为独立“哲学层”出现，它只回答 State 中的一个问题：

一个用户可见的名字、句柄或逻辑地址，通过哪些中间关系最终引用哪个对象？

典型实例：

$\text{path} \rightarrow \text{directory/dentry relation} \rightarrow \text{inode},$
 $\text{fd} \rightarrow \text{open-file reference} \rightarrow \text{file object},$
 $\text{VPN} \rightarrow \text{page-table mapping} \rightarrow \text{PFN},$
 $\text{file offset} \rightarrow \text{block mapping} \rightarrow \text{storage block}.$

间接层真正买来的东西

增加一层关系查找会增加成本，但换来：**重定位、共享、隔离、重命名、稀疏表示、延迟绑定、权限检查、生命周期解耦。**

因此 Indirection 的本质不是“多查一张表”，而是：

用一次额外查找换取对象与名字/位置/实现之间的解耦。

关系改变 对象改变

最经典例子是：

`unlink(path).`

它首先删除的是：

$\text{Name} \dashrightarrow \text{inode}$

这条命名关系；如果仍存在：

$\text{fd} \dashrightarrow \text{OpenFile} \dashrightarrow \text{inode},$

对象仍可继续使用。

因此所有生命周期题都应先画引用图，而不是背“删除文件”的口号。

第 3 章：Queues——稀缺和等待在状态中的统一表示

为什么 Queue 属于 State，而不是“调度算法”

队列描述的是当前有哪些请求正在等待某个条件或资源。它本身是一部分系统状态，而不是策略。

队列/等待结构	等待什么	谁/什么事件使其离开
Run Queue	CPU 时间	scheduler dispatch
Lock Wait Queue	锁/条件成立	unlock / signal / wakeup
I/O Request Queue	设备服务能力	device dispatch / completion
Pipe Waiters	buffer 非空/非满	peer read / write
Reclaim/Writeback	内存/脏页压力释放	reclaim / writeback completion

Scarcity 不再单独成章

所谓“有限资源下的仲裁”可以严格拆成两个已经存在的元素：

Scarcity $\Rightarrow$ Queue state appears

和

Arbitration $\Rightarrow$ Policy selects a waiter/request.

所以“Scarcity/Arbitration”不需要再做一条 Law。它是 **State** 的 **Queue + Transition** 的 **Policy** 组合出来的现象。

需求大于供给时的通用状态变化

$$\text{Request} \rightarrow \begin{cases} \text{Grant}, & \text{Capacity} > 0, \\ \text{Enqueue/Block/Reject}, & \text{Capacity} = 0. \end{cases}$$

资源释放后再由策略决定：

$$\text{Release} \rightarrow \text{Choose Waiter} \rightarrow \text{Wake/Grant}.$$

第 4 章：Lifetime——它不是第四种结构，而是对象与关系随时间的变化

生命周期不需要与 Objects/Relations/Queues 并列成第四种 State 元素。它本质上是：

- 对象何时加入或移出 O ；
- 引用边何时加入或移出 R ；
- 哪些生命周期不变量必须始终成立。

典型模式：

$Create \rightarrow AddReference \rightarrow Use \rightarrow DropReference \rightarrow Destroy$

但 Destroy 的条件不是“调用了 delete/close/free”，而是某个生命周期不变量成立，例如：

No reachable/open reference $\wedge$ object no longer named/owned.

高频错误

逻辑删除不等于立即销毁。unlink()、close()、进程退出、page unmap 都可能只删除某一类关系；对象能否真正回收，要看剩余引用、所有权和子系统自己的生命周期规则。

Part II: Transition——内核状态到底怎样变化

第 5 章：Event——状态变化的触发输入

内核是“事件驱动”的准确含义

本册只把 Event 定义为：触发内核重新评估并可能改变状态的输入。典型来源：

- 程序主动：syscall、lock/unlock、exit；
- 当前指令同步产生：page fault、protection fault、illegal instruction；
- 硬件异步产生：timer、I/O completion interrupt；
- 内核内部产生：memory pressure、writeback threshold、timeout、wakeup；
- 故障产生：process crash、system crash、I/O error。

因此“事件驱动”不再另外成为一条 Law，它就是 Transition 的输入 E_t 。

事件识别法

综合题中先圈出每一个真正改变状态的事件。例如：read() 调用、page-cache miss、进程睡眠、DMA completion、interrupt、wakeup、scheduler dispatch。不要把“执行了一段代码”笼统看成一个事件。

第 6 章：Mechanism——定义“允许怎么变”

机制回答：系统具备哪些受控状态转换能力？

机制	它允许的状态变化	它不负责决定什么
Trap/System-call entry	User execution → Kernel execution	具体服务策略
Context Switch	CPU context A → B	下一次选 B 还是 C
Page Fault Handler	nonresident / permission fault → repair or reject	victim / allocation policy
Atomic RMW	+ lock state / waiter state 可原子改变	谁应该先拿锁的公平策略
Sleep/Wakeup		
Block I/O + DMA	request 可提交并异步完成	多请求先服务谁
Journal/Replay	多步持久更新可记录、提交、恢复	哪些高层操作何时触发持久化

第 7 章：Policy——只在“存在选择”时出现

策略不是所有状态转换都需要。只有当机制给出了多个合法候选动作时，Policy 才有意义：

$$a_t = \pi(S_t, E_t, \mathcal{A}_t).$$

典型：

- scheduler：多个 Ready task 选谁；
- page replacement：多个可回收页选谁；
- I/O scheduling：多个 pending request 先发谁；

- lock fairness: 多个 waiter 怎样排序;
- allocation: 多个可用 frame/block/extent 选哪个。

Mechanism / Policy 的边界

Mechanism = 定义合法动作集合

Policy = 从合法动作集合中选择

如果只有一个合法动作，就没有策略问题；如果连动作都做不到，说明缺的是机制，不是策略。

为什么这能消除 “Law 3 vs Loop 2” 的重复

“有限资源下的仲裁”现在不再是第三套理论：

1. State 中已经存在一个 Queue；
2. Event 表示资源释放或请求到达；
3. Mechanism 提供 enqueue/dequeue/block/wakeup 能力；
4. Policy 决定从 Queue 中选谁；
5. Invariant 检查不能重复授予、不能越权；
6. Cost 再考虑公平性、等待时间和吞吐。

因此没有必要再定义一个独立“Resource Arbitration Loop”。

Part III: Evaluation——什么叫“正确”，什么叫“更好”

第 8 章：Invariants——保护、一致性、进展都属于合法性约束

不变量的统一定义

状态转换以后必须满足：

$$I(S_{t+1}) = \text{true}.$$

所谓 Protection、Isolation、Atomicity、Consistency 并不是四套独立架构，它们都是对合法状态/合法转换的不同约束。

五类常见不变量

不变量类型	核心问题	典型例子
Safety	什么绝不能发生？	同一独占资源不能同时授予两个 owner
Protection/Isolation	谁能对哪个对象做什么？	User 不能写 kernel page；fd 权限必须合法
Progress/Liveness	合法请求是否最终可能继续？	无死锁、无永久丢失唤醒；公平性视策略而定
Structural Consistency	对象/关系是否自治？	inode / block bitmap、引用计数、队列状态一致
Persistence/Crash Consistency	故障后能否恢复到合法持久状态？	journal commit / replay 后 metadata 一致

Protection 的唯一住所

保护在本册不再既当“OS 职责”又当“运行 Law”重复出现。统一写成约束：

$$\text{Allowed}(\text{Subject}, \text{Operation}, \text{Object}, \text{Authority}).$$

任何状态转换只要违反这个谓词，就必须被硬件或内核拒绝、陷入、返回错误或终止。

并发一致性与崩溃一致性的统一

二者共同研究：一个逻辑操作被部分执行时，非法中间状态会不会暴露或永久留下。

并发：

$$A \longrightarrow B \longrightarrow C$$

可能变成：

$$A \longrightarrow \boxed{\text{Interleaving}} \longrightarrow B \longrightarrow C.$$

崩溃：

$$W_1 \longrightarrow W_2 \longrightarrow W_3$$

可能变成：

$$W_1 \longrightarrow W_2 \longrightarrow \boxed{\text{Crash}}.$$

二者的共同目标都是：

$$\boxed{\text{让外部可观察状态始终满足关键不变量，或能恢复到满足不变量的状态。}}$$

手段不同：并发主要使用原子操作、锁、条件同步和有序提交；持久化主要使用 write ordering、journal/WAL、commit、replay、repair。

第 9 章：Cost——性能原则只放在这一层

统一成本函数

正确状态不止一个，因此还要关心：

$$C = Latency + CPU + Space + MemoryTraffic + IO + Contention + FairnessPenalty + RecoveryCost.$$

不同机制优化的是不同项，不能把“更快”当作单一目标。

Fast / 慢路径的唯一位置

快慢路径不再是运行 Law，而是成本优化模式：

$$CommonCase \rightarrow CheapFastPath$$

$$Miss/Conflict/Pressure \rightarrow RareComplexSlowPath.$$

子系统	快路径	慢路径
Virtual Memory	TLB hit / resident mapping	page-table walk / page fault / I/O
File System	dentry/page-cache hit	path miss / block mapping / storage I/O
Lock	uncontended atomic acquire	contention $\rightarrow$ wait queue / scheduler
I/O	buffered/cache hit / async device work	queueing / retry / error / wakeup
Process	direct user execution	syscall/exception/interrupt enters kernel

Locality、Caching、Laziness、Batching 不是新“原则层”

这些都只是降低 C 的策略/技术：

- **Locality**：预测近期再次访问或访问邻近对象；
- **Caching**：保存慢对象的快速副本；
- **Laziness**：把工作推迟到真正需要时，如 demand paging、COW、delayed allocation；
- **Batching**：把多个小动作合并，摊薄固定开销，如 writeback、I/O merging；
- **Speculation/Prefetch**：提前做可能需要的工作，换取平均延迟下降。

优化不能破坏不变量

所有性能机制必须服从：

$$Correctness\ first,\ optimization\ second.$$

COW 可以延迟复制，但不能破坏进程隔离；writeback 可以延迟持久化，但必须明确 durability 边界；lock 快路径可以省掉调度，但不能破坏互斥。

Part IV：从模型到题目——五种轨迹模板

第 10 章：五种常见轨迹模板

使用方法

五种轨迹只描述常见时间结构。每次使用仍需填写 O, R, Q, E, M, π, I, C ，不能用轨迹名称替代状态分析。

轨迹 A：Resolve——名字/句柄/逻辑地址解析

$$Name/Handle \rightarrow RelationLookup \rightarrow Object$$

例如：

$$fd \rightarrow OpenFile, \quad VPN \rightarrow PTE \rightarrow Frame, \quad Path \rightarrow Dentry/Inode.$$

这类题首先画 R ，一般不涉及 Queue；若查找 miss/fault，再进入其他轨迹。

轨迹 B：Wait / Wake——条件暂不满足

$$Running \rightarrow Check \rightarrow Block/Enqueue \rightarrow Event \rightarrow Wake \rightarrow Ready$$

例如锁竞争、阻塞 I/O、pipe empty/full、条件变量。核心状态是 Q 和 task state；核心转换机制是 atomic sleep/wakeup；策略只在多个 waiter 中选择时出现。

轨迹 C：Fault / Repair / Retry——映射或条件暂时不能满足

$$Attempt \rightarrow Fault/Miss \rightarrow KernelRepair \rightarrow UpdateState \rightarrow Retry$$

典型：page fault、COW、file-backed mmap 首次访问。注意这不是“Mapping Loop”新理论，只是某次 Transition 的时间结构。

轨迹 D：Pressure / Reclaim——需求超过容量

$$Demand > Capacity \rightarrow Pressure \rightarrow SelectVictim \rightarrow Reclaim/Writeback \rightarrow Retry$$

这里的 SelectVictim 才是 Policy；reclaim/writeback 是 Mechanism；不能把整个过程笼统叫“页面置换算法”。

轨迹 E: Modify / Commit / Recover——持久状态的多步更新

Modify → Dirty/Pending → Commit → Stable

若故障:

Crash → InspectLog/Metadata → Replay/Repair → ConsistentState.

它用于区分“内存可见”“系统调用返回”“设备完成”“真正持久”这些不同状态。

第 11 章：五大子系统在唯一模型中的投影

子系统	Objects	Relations	Queues
Process/Sched	task / thread	task→mm/files	run / wait queue
VM	VMA / PTE / page	VA→PTE→frame	reclaim / writeback wait
Concurrency	lock / semaphore / waiter	owner / wait relation	lock / CV wait queue
I/O	device / request / buffer	request→device	request / completion queue
File System	fd / file / inode / dentry	path / fd / offset mappings	writeback / cache-related waits

子系统	Events	Mechanism / Policy	Invariant / Cost
Process/Sched	timer / syscall / exit	switch, block, wake / scheduling	execution correctness / response, fairness
VM	fault / pressure / unmap	map, fault, reclaim / replacement	isolation / miss, I/O, space
Concurrency	lock / unlock / signal	atomic + sleep/wake / fairness	mutual exclusion, order / contention
I/O	submit / interrupt / error	DMA, interrupt, driver / request order	request correctness / latency, BW, CPU
File System	open / read / write / unlink / crash	VFS, block map, journal / allocation	namespace, crash consistency / lookup, durability

看矩阵的方法

不要横着背整行。综合题应当竖着读一列，再跨列跟踪同一个事件。例如一个 blocking read 会先修改 Process 列的 task state，再进入 File System 列解析对象，再进入 I/O 列提交请求，完成后又回到 Process 列 wakeup。

Part V：九个跨专题过程

第 12 章：八项分析表

任何跨专题综合题都填写下面八项：

1. **Objects**：涉及哪些对象？
2. **Relations**：谁指向谁、映射到谁？
3. **Queues**：谁在等什么？
4. **Event**：当前真正发生了什么触发事件？
5. **Mechanism**：系统用什么能力改变状态？
6. **Policy**：哪里存在“选谁/选哪个”的选择？没有就写“无”。
7. **Invariant**：什么绝不能被破坏？
8. **Cost**：主要成本在哪里？有没有 fast/慢路径？

为什么只用八槽

八槽与统一模型一一对应，不增加任何新分类。综合题的复杂度只来自**对象多、事件多、跨多个子系统连续变化**，而不是需要额外学习另一套“综合理论”。

第 13 章：综合题一——程序从 fork 到 exec、运行、exit、wait

核心轨迹

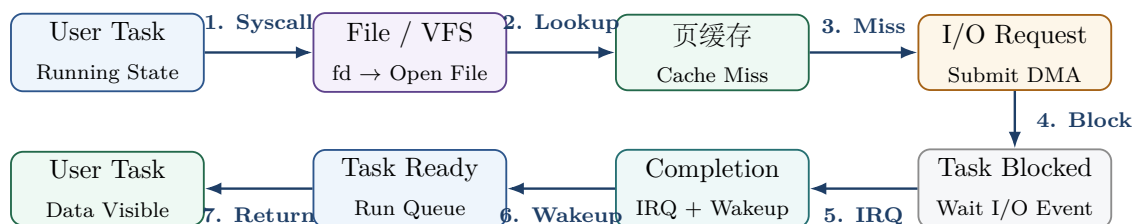
$$\begin{aligned}
 &Parent \xrightarrow{fork} Parent + Child \\
 &\quad \xrightarrow{exec} Child(NewAddressSpace) \\
 &\quad \xrightarrow{run/syscall/fault} RunningSystem \\
 &\quad \xrightarrow{exit} Zombie/ExitState \xrightarrow{wait} Reaped.
 \end{aligned}$$

槽位	本题关键内容
Objects	parent task、child task、address space、page tables、open files
Relations	child 初始继承/共享部分资源关系；exec 替换程序映像；fd 关系通常按规则保留
Queues	child/parent 进入 run queue；wait 时 parent 可能进入 wait queue
Events	fork、exec、timer、fault、exit、wait
Mechanism	task create、COW、context switch、address-space replace、wake/reap
Policy	scheduler 选择谁运行
Invariant	地址空间隔离；父子资源引用合法；退出状态不被过早释放
Cost	COW 减少 fork 初始复制；fault 把复制成本推迟到写入慢路径

第 14 章：综合题二——一次 blocking read(fd,buf,n)

四条信息同时追踪

综合版不再只画一条调用栈，而同时追踪：**Control**、**Task State**、**Data**、**Reference**。



槽位	本题关键内容
Objects	task、fd table、open file、inode、page-cache page、I/O request、device
Relations	fd→file→inode；file/page-index→ 页缓存；request→device
Queues	task I/O wait queue、run queue、device request queue
Events	syscall、cache miss、submit、block、DMA completion、interrupt、wakeup、dispatch
Mechanism	VFS lookup、block mapping、DMA、interrupt、sleep/wakeup、context switch
Policy	scheduler；若多个 I/O 请求则 request ordering
Invariant	user buffer 权限合法；request 只能正确完成一次；blocked/ready 状态一致
Cost	page-cache hit 是快路径；miss 进入 storage 慢路径；copy 与 I/O 是主要成本

第 15 章：综合题三——file-backed mmap 的首次访问

`mmap(file)` → *VMA created* → *first load/store*
 → *page fault* → *file page resolve*
 → *I/O?* → *PTE update* → *retry*.

槽位	本题关键内容
Objects	task、VMA、PTE、file/inode、page-cache page、physical frame
Relations	VA→VMA；VMA→file+offset；PTE→frame；file page→ 页缓存
Queues	cache miss 时可能出现 I/O wait
Events	mmap syscall、first access、page fault、I/O completion
Mechanism	fault handling、file-backed page lookup、PTE install、instruction retry
Policy	frame allocation/reclaim 可能涉及选择；普通映射命中时无策略
Invariant	权限、文件映射范围、进程隔离正确
Cost	lazy mapping 把实际 I/O 推迟到 first-touch 慢路径；后续访问回到 MMU 快路径

第 16 章：综合题四——fork + COW + 文件描述符共享

同一次 `fork()` 后，两类关系具有完全不同的语义：

Memory : *separate logical address spaces* + *temporarily shared frames*

File : separate fd tables + references to same open file description

因此 child 写内存时:

$COW\ fault \rightarrow new\ frame \rightarrow child\ PTE\ update,$

而父子对共享 OFD 的 `read/lseek` 可能共同改变同一个 offset。

跨专题识别点

题目问“fork 以后是否共享”时禁止回答“共享/不共享”两个字。必须先问：**共享的是哪个对象或哪条关系？** Address space identity、physical frame、fd table、打开文件描述、inode 都不是同一层。

第 17 章：综合题五——Mutex contention

$T_1 : acquire \rightarrow owner,$

$T_2 : acquire \rightarrow atomic\ fail \rightarrow wait\ queue \rightarrow Blocked.$

随后:

$T_1 : unlock \rightarrow wake(T_2) \rightarrow Ready \rightarrow scheduler \rightarrow Running.$

槽位	本题关键内容
Objects	T1、T2、mutex/lock object
Relations	owner relation: lock→T1; waiter relation: T2→lock
Queues	lock wait queue + run queue
Events	acquire、atomic failure、sleep、unlock、wakeup、dispatch
Mechanism	atomic RMW、sleep/wakeup、scheduler
Policy	多个 waiter 时可能有公平/优先级选择
Invariant	mutual exclusion; 不能 lost wakeup; owner/lifetime 合法
Cost	uncontended 快路径 vs contended block/schedule 慢路径

第 18 章：综合题六——Pipe empty/full 与背压

Pipe 是非常纯粹的跨专题对象: 它既是 fd 可引用的内核对象, 又有 buffer state、reader/writer waiters 和 blocking semantics。

$Reader + Empty\ Pipe \rightarrow Blocked,$

$Writer\ writes \rightarrow BufferStateChange \rightarrow WakeReader.$

反向:

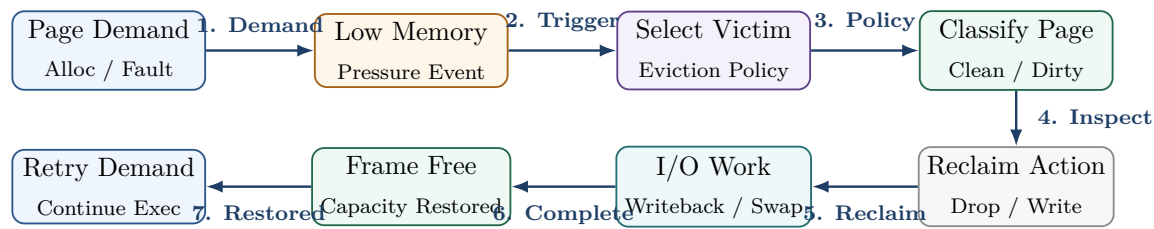
$Writer + Full\ Pipe \rightarrow Blocked,$

$Reader\ consumes \rightarrow FreeSpace \rightarrow WakeWriter.$

所谓 backpressure 不需要再成为独立 Law, 它只是本题状态机中的结果:

$Demand > BufferCapacity \Rightarrow Enqueue/Block.$

第 19 章：综合题七——Memory Pressure 的一生



这道综合题把 VM、页缓存、File System、I/O 与 Scheduling 接起来：

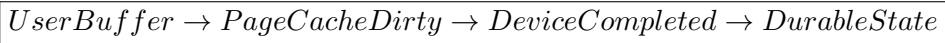
- clean file-backed page：可丢弃，未来从文件恢复；
- dirty file-backed page：通常先 writeback，进入 I/O；
- anonymous page：若要回收，可能依赖 swap 或其他机制；
- 如果 reclaim/I/O 不能立即完成，当前 task 可能等待。

最关键的分工

Pressure 是事件/状态条件；**Victim selection** 是 Policy；**unmap/writeback/swap/reclaim** 是 Mechanism；不能回收仍被必要引用或 **pinned** 的页属于 Invariant；最终回收延迟和 I/O 是 Cost。

第 20 章：综合题八——write + fsync + crash

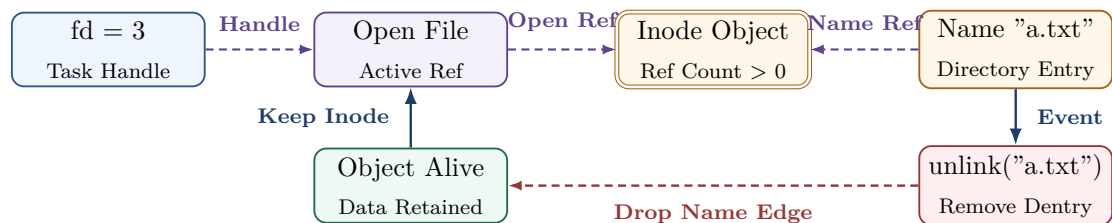
必须区分四个状态：



它们不是同一个“写完”。

槽位	本题关键内容
Objects	file/inode、dirty pages、journal transaction、block requests、device cache
Relations	file offset→ 页缓存；metadata→journal/blocks
Queues	writeback/I/O request queue
Events	write、dirty threshold、fsync、I/O completion、crash、recovery
Mechanism	dirty marking、writeback、journal commit、flush/FUA/设备完成、replay
Policy	writeback timing / allocation / request scheduling 等可能存在策略
Invariant	crash 后 metadata/namespace 必须可恢复到合法状态；fsync 语义必须满足
Cost	延迟写降低前台延迟，却把持久化成本推迟到 writeback/fsync 慢路径

第 21 章：综合题九——unlink while open



本题只要坚持 State 模型就不会错: `unlink()` 删除一条 Naming relation, 不等于立刻从 Objects 集合删除 inode/file data。真正回收要等剩余引用关系满足生命周期不变量。

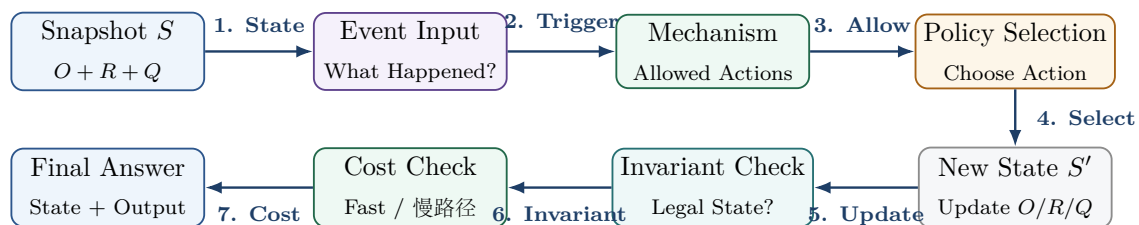
Part VI: 408 综合题统一协议

第 22 章：跨专题十六问

拿到综合题不要先回忆“这是哪一章”。按顺序问：

1. 当前有哪些 **Objects**?
2. 当前各对象处于什么 **State**?
3. 有哪些 **Relations/Mapping**?
4. 是否存在 **Queue/Waiter**?
5. 当前真正发生了什么 **Event**?
6. CPU 此刻在 User 还是 Kernel?
7. 哪个 **Mechanism** 允许这次状态转换?
8. 是否存在多个合法候选动作? 若有, 哪个 **Policy** 选择?
9. 哪条 **Protection/Safety Invariant** 必须保持?
10. 是否涉及 **Progress/Liveness**? 谁能制造继续运行的事件?
11. 是否涉及对象 **Lifetime**? 哪些引用被创建/删除?
12. 数据此刻在哪里? 是否在 cache/buffer/device 中移动?
13. 是否发生 fast-path miss, 进入慢路径?
14. 如果 Demand > Capacity, 谁排队、阻塞、回收或被拒绝?
15. 如果中途 crash/interleaving, 是否留下非法部分状态?
16. 最终题目问的是哪个 Cost: 时间、空间、I/O、公平、竞争还是持久性?

第 23 章：一页式综合决策树



核心结论

$$\text{Snapshot } S = (\text{Objects}, \text{Relations}, \text{Queues})$$

$$\text{Event} \rightarrow \text{Mechanism} \rightarrow \text{Policy}(\text{if needed}) \rightarrow S'$$

$$\text{Invariant}(S') = \text{true} \quad \text{and} \quad \text{Cost is acceptable}$$

进程、虚拟内存、并发、I/O、文件系统的差异，体现在对象、关系、队列、事件、机制、策略、不变量和成本函数不同；分析步骤保持一致。

附录 A：概念归属检查表

概念	正式出现位置	遇到相关题时怎么用
Naming/Indirection	Relations	画出逻辑名字/句柄到对象的边
Protection/Isolation	Invariants	检查“主体-操作-对象”是否允许
Mechanism/Policy	Transition	先列合法动作，再判断是否需要选择
Scarcity/Arbitration	Queue + Policy	先确认是否排队，再问策略选谁
Backpressure	Queue 状态变化	供给不足时分析阻塞、排队、拒绝或回收
Consistency/Recovery	Invariant + repair	找出部分完成状态与修复路径
Fast/慢路径	Cost	比较常见路径与 miss/conflict 的成本差
Event-driven	Event	找真正触发状态改变的输入
Lifetime/Ownership	O/R 演化 + 约束	追踪引用增减与最终销毁条件
Caching/Lazy/Batching	Cost optimization	解释为何推迟、复用或合并工作

附录 B：专题与综合过程的边界

- **进程专题**负责讲清 task/process/thread、调度算法、trap/context switch 的局部机制；综合篇只追踪它们怎样与其他对象共同变化。
- **并发/PV 专题**负责同步原语和经典模型；综合篇只抽象为 lock object、wait queue、event、invariant。
- **虚拟内存专题**负责页表、TLB、fault 与 replacement；综合篇只追踪 VM 事件怎样继续进入文件、I/O 与进程状态。
- **I/O 专题**负责 polling、interrupt、DMA、driver、页缓存与 spooling；综合篇只追踪 request、block、completion 与 wake。
- **文件系统专题**负责 path/fd/inode/block/journal；综合篇只关注这些关系怎样跨 VM/I/O/进程状态传播。

使用边界

专题册负责定义机制；综合册负责让多个机制在同一状态机中协作。因此这里不重复定义页表、PV、inode 或 DMA，只训练跨对象、跨事件和跨不变量的推演。